CCNA 200-301 · EPISODE 1

OSI MODEL DEEP DIVE

7 Layers · Real Troubleshooting · CLI Commands

⏱ 22 Min Deep Dive @jahircyberdesk

7	Application	HTTP · DNS · SMTP
6	Presentation	SSL · TLS · JPEG
5	Session	NetBIOS · RPC
4	Transport	TCP · UDP · Ports
3	Network	IP · Router · BGP
2	Data Link	MAC · Switch · VLAN
1	Physical	Cable · Hub · Bits

FOUNDATION

What Is the OSI Model?

The **Open Systems Interconnection (OSI)** model is a **conceptual framework** that standardizes how different network systems communicate — regardless of vendor or technology.

📅 HISTORY

- **1984** — Published by ISO (ISO 7498)
- **1994** — Revised as ISO/IEC 7498-1
- Created to end **vendor lock-in**
- Still the gold standard for **network education**

🎯 PURPOSE

- Universal **troubleshooting language**
- Divide network into **7 manageable layers**
- Each layer has a defined responsibility
- Vendors can build **interoperable** products

MEMORY TRICK

"All People Seem To Need Data Processing"

7	Application	All
6	Presentation	People
5	Session	Seem
4	Transport	To
3	Network	Need
2	Data Link	Data
1	Physical	Processing

WHY IT MATTERS

Why Every IT Admin Needs OSI

🔧 TROUBLESHOOTING POWER

- Narrow a fault to the **exact layer** — stop guessing
- L1 issue? Check cables before rebooting servers
- L3 issue? Ping gateway before blaming the app
- Cuts **MTTR by 60%** when used systematically

🗣 COMMUNICATION

- Speak the **same language** as vendors & engineers
- "It's a Layer 2 loop" — everyone knows what to fix
- Essential for **CCNA, CCNP, CompTIA** exams
- Required for **senior sysadmin** roles

📋 REAL SCENARIOS

- User can't browse → **start at L1, work up**
- Firewall blocking? That's **L3/L4**
- SSL cert error? That's **L6**
- App timeout? Could be **L4 or L7**

🏅 CAREER IMPACT

- Foundation for **every certification**
- Used daily in **NOC & help desk** environments
- Required knowledge for **network design**
- Separates juniors from **senior engineers**

OVERVIEW

All 7 Layers at a Glance

Layer	Name	PDU	Key Protocols	Devices	Admin Focus
7	**Application**	Data	HTTP, DNS, SMTP, FTP	None	App behavior
6	**Presentation**	Data	SSL/TLS, MIME, JPEG	None	Encryption, encoding
5	**Session**	Data	NetBIOS, RPC, SIP	None	Session tracking
4	**Transport**	Segment	TCP, UDP	None	Port, reliability
3	**Network**	Packet	IP, OSPF, BGP	Router	IP routing
2	**Data Link**	Frame	Ethernet, 802.1Q	Switch	MAC, VLANs
1	**Physical**	Bits	Ethernet PHY, DSL	Hub, NIC	Cables, signals

LAYER 1 — PHYSICAL

Physical Layer Fundamentals

Job: Transmit raw **bits (0s and 1s)** over a physical medium. It defines the electrical, optical, and mechanical characteristics.

- Operates with **bits** — no concept of addressing
- Defines cable types: **UTP, Coax, Fiber**
- Defines connector standards: **RJ-45, SC, LC**
- Sets signal levels, timing, and **encoding schemes**
- Governs duplex: **half-duplex vs full-duplex**

📡 TRANSMISSION MEDIA

- **UTP Cat5e/Cat6** — up to 1 Gbps / 10 Gbps
- **Single-mode fiber** — up to 100 km, WAN links
- **Multi-mode fiber** — short range, data centers
- **Coaxial** — legacy, DOCSIS cable internet
- **Wireless (RF)** — 802.11 a/b/g/n/ac/ax

🔢 ENCODING SCHEMES

- **Manchester Encoding** — 10Base-T Ethernet
- **4B/5B + NRZI** — Fast Ethernet (100Base-TX)
- **8B/10B** — Gigabit Ethernet (1000Base-T)

LAYER 1 — PHYSICAL

Devices, Standards & CLI Commands

PHYSICAL LAYER DEVICES

- **Hub** — broadcasts all traffic to all ports (legacy)
- **Repeater** — regenerates signal over long distances
- **NIC** — Network Interface Card, encodes/decodes frames
- **Media Converter** — copper ↔ fiber signal conversion
- **Transceiver (SFP/QSFP)** — pluggable optical

L1 TROUBLESHOOTING CLI

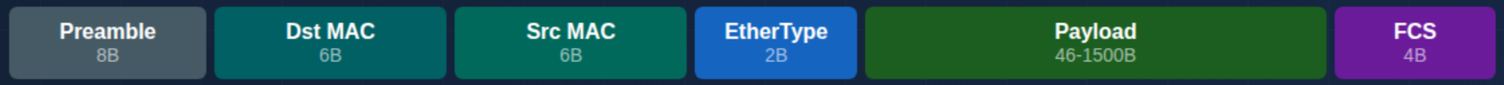
```
# Check interface physical status (Cisco IOS) show interfaces GigabitEthernet0/0 GigabitEthernet0/0 is up, line protocol is up Hardware is ISR4321, address is aabb.cc00.0100 Full-duplex, 1000Mbps, link type is auto # Check interface errors show interfaces GigabitEthernet0/0 counters errors CRC: 0 InputErrors: 0 Giants: 0 # Linux: check NIC link status ethtool eth0 Speed: 1000Mb/s Duplex: Full Link detected: yes # Check cable errors on Linux ip link show eth0 dmesg | grep eth0
```

LAYER 2 — DATA LINK

Data Link Layer Fundamentals

Job: Node-to-node delivery of **frames** on the same network segment. Provides error detection and MAC addressing.

📦 ETHERNET FRAME STRUCTURE



🔀 TWO SUBLAYERS (IEEE 802)

- **LLC (802.2)** — Logical Link Control: identifies upper-layer protocol, flow control
- **MAC (802.3)** — Media Access Control: MAC addressing, CSMA/CD, frame delimiting

🏷 MAC ADDRESS FACTS

- 48-bit address — **AA:BB:CC:DD:EE:FF**
- First 24 bits = **OUI** (vendor identifier)
- Last 24 bits = **device identifier**
- Burned in at factory → **BIA**
- Can be **overridden** in software
- **FF:FF:FF:FF:FF:FF** = broadcast

🔍 ERROR DETECTION

- **FCS (CRC-32)** — detects bit errors in frame
- L2 detects errors — **doesn't correct them**
- Corrupted frames are **silently dropped**

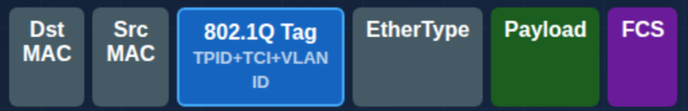
LAYER 2 — DATA LINK

VLANs & Switching Deep Dive

🔀 HOW A SWITCH WORKS

- Builds a **MAC address table** (CAM table) by learning source MACs
- Forwards frames to the **specific port** where destination MAC lives
- Unknown destination → **floods** all ports in VLAN
- Runs **STP (802.1D)** to prevent loops

🏷 802.1Q VLAN TAGGING



- VLAN ID = **12 bits** → VLANs 1–4094
- **Access port** — carries one VLAN, tag stripped
- **Trunk port** — carries multiple VLANs with 802.1Q tags

🖥 L2 SWITCHING CLI (CISCO)

```
# Show MAC address table show mac address-table # Create VLAN and name it vlan 10 name SALES # Configure access port interface Fa0/1 switchport mode access switchport access vlan 10 # Configure trunk port interface Gi0/1 switchport mode trunk switchport trunk allowed vlan 10,20,30 # Verify show vlan brief show interfaces trunk
```

LAYER 3 — NETWORK

Network Layer Fundamentals

Job: Route **packets** between different networks using logical IP addressing. The "post office" of networking.

📦 IP PACKET HEADER (KEY FIELDS)

Field	Size	Purpose
Version	4 bits	IPv4 (4) or IPv6 (6)
TTL / Hop Limit	8 bits	Prevents routing loops
Protocol	8 bits	TCP=6, UDP=17, ICMP=1
Src / Dst IP	32 bits each	Source and destination address
Header Checksum	16 bits	Header integrity check

🔀 HOW A ROUTER WORKS

- Receives packet → checks **destination IP**
- Looks up **routing table** for best match
- Decrements **TTL** by 1 (drop if TTL=0)
- Strips L2 frame, **re-encapsulates** for next hop
- Forwards out the **best-match interface**

📡 KEY L3 PROTOCOLS

- **IP (IPv4 / IPv6)** — addressing & routing
- **ICMP** — ping, traceroute, unreachable messages
- **ARP** — resolves IP → MAC (technically L2/L3 boundary)
- **NAT** — translates private ↔ public IPs

LAYER 3 — NETWORK

IP Addressing Deep Dive

🌐 IPV4 CLASSES & PRIVATE RANGES

Range	Class	Use	CIDR
10.0.0.0	A	Private (large orgs)	/8
172.16–31.x	B	Private (medium)	/12
192.168.x.x	C	Private (home/SOHO)	/24
127.0.0.1	—	Loopback	/8
169.254.x.x	—	APIPA (no DHCP)	/16

✂️ CIDR SUBNETTING QUICK REF

CIDR	Subnet Mask	Hosts
/24	255.255.255.0	**254**
/25	255.255.255.128	**126**
/26	255.255.255.192	**62**
/30	255.255.255.252	**2** (P2P links)

⚡ IPV6 KEY FACTS

- **128-bit** address — 2001:db8::1/64
- **::1** = IPv6 loopback (equivalent to 127.0.0.1)
- **fe80::/10** = Link-local (auto-configured)
- **No broadcast** — replaced by multicast
- **NDP** replaces ARP (Neighbor Discovery Protocol)
- EUI-64 generates interface ID from **MAC address**

🖥 IP CLI COMMANDS

```
ip addr show # Linux IP config ipconfig /all # Windows IP config ping 8.8.8.8 # Test L3 connectivity traceroute 8.8.8.8 # Trace routing path show ip interface brief # Cisco IOS summary
```

LAYER 3 — NETWORK

Routing Protocols Deep Dive

Protocol	Type	AD	Algorithm	Use Case
Static	Manual	1	None	Small / stub nets
OSPF	Link-State	110	Dijkstra SPF	Enterprise IGP
EIGRP	Hybrid	90	DUAL	Cisco enterprise
RIP v2	Distance-Vec	120	Bellman-Ford	Lab / legacy
BGP	Path-Vector	20/200	Best-Path	Internet / ISP

📌 ADMINISTRATIVE DISTANCE (AD) — LOWER IS BETTER

Connected=**0** → Static=**1** → EIGRP=**90** → OSPF=**110** → RIP=**120** → External BGP=**20**

🖥 ROUTING CLI (CISCO IOS)

```
# Show routing table show ip route C 192.168.1.0/24 is directly connected, Gi0/0 O 10.0.0.0/8 [110/2] via 192.168.1.1, Gi0/0 S* 0.0.0.0/0 [1/0] via 203.0.113.1 # Add static default route ip route 0.0.0.0 0.0.0.0 203.0.113.1 # Enable OSPF router ospf 1 network 192.168.1.0 0.0.0.255 area 0 # Verify OSPF neighbors show ip ospf neighbor show ip ospf database # Linux routing ip route show ip route add 10.0.0.0/8 via 192.168.1.1
```

LAYER 4 — TRANSPORT

Transport Layer Fundamentals

Job: End-to-end communication between **processes** (not just hosts). Provides segmentation, port numbers, and optionally reliability.

📦 TCP SEGMENT HEADER (KEY FIELDS)

Field	Purpose
Source / Dest Port	Identifies sending/receiving application
Sequence Number	Tracks byte order for reassembly
Ack Number	Next expected byte from other side
Flags	SYN, ACK, FIN, RST, URG, PSH
Window Size	Flow control — receiver buffer size

🔌 PORT NUMBER RANGES

Range	Type	Example
0–1023	**Well-Known**	HTTP=80, HTTPS=443, SSH=22
1024–49151	**Registered**	MySQL=3306, RDP=3389
49152–65535	**Ephemeral**	Client source ports

🖥 TRANSPORT CLI

```
# Show active connections ss -tunap # Linux netstat -ano # Windows # Test TCP port nc -zv 192.168.1.1 443 # Test UDP port nc -zuv 192.168.1.1 53
```

LAYER 4 — TRANSPORT

TCP 3-Way Handshake

1. **SYN** — Client sends segment with SYN flag set. SEQ=x (random). "I want to connect."
2. **SYN-ACK** — Server responds. SYN+ACK flags. SEQ=y, ACK=x+1. "OK, I'm ready."
3. **ACK** — Client acknowledges. ACK=y+1. Connection ESTABLISHED.

Data Transfer

⚡ FLOW & CONGESTION CONTROL

- **Sliding Window** — receiver advertises window size (how much buffer remains)
- **Slow Start** — TCP begins sending 1 MSS, doubles until congestion threshold
- **Fast Retransmit** — 3 duplicate ACKs triggers immediate retransmit (no wait)
- **Nagle Algorithm** — buffers small segments to reduce overhead

🖥 CAPTURE & DEBUG

```
# Capture TCP handshake tcpdump -i eth0 'tcp[tcpflags] & tcp-syn != 0' # Show TCP state ss -tn state established ss
-tn state time-wait # Wireshark filter tcp.flags.syn == 1 and tcp.flags.ack == 0
```

LAYER 4 — TRANSPORT

TCP vs UDP + Port Reference

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Guaranteed delivery, retransmission	Best-effort, no retransmit
Ordering	Sequence numbers ensure order	No ordering
Flow Control	Window size, congestion control	None
Speed	Slower (overhead)	Faster (low overhead)
Use Cases	HTTP, SSH, FTP, SMTP, RDP	DNS, DHCP, VoIP, Gaming, TFTP
Header Size	20–60 bytes	8 bytes

🔌 ESSENTIAL PORT NUMBERS

Port	Protocol	Transport	Use
22	SSH	TCP	Secure shell
23	Telnet	TCP	Insecure shell (avoid!)
25	SMTP	TCP	Email send
53	DNS	TCP/UDP	Name resolution
67/68	DHCP	UDP	IP assignment
80	HTTP	TCP	Web (plain)
443	HTTPS	TCP	Web (encrypted)
3389	RDP	TCP	Remote Desktop
3306	MySQL	TCP	Database

LAYER 5 — SESSION

Session Layer Deep Dive

Job: Establish, manage, and terminate **sessions** between applications. Handles authentication, synchronization, and dialog control.

🔄 SESSION LIFECYCLE

ESTABLISH — Auth & setup → **MAINTAIN** — Data transfer → **CHECKPOINT** — Sync point saved →

TERMINATE — Graceful close

🔐 CHECKPOINTING & RECOVERY

- Saves **synchronization points** during long data transfers
- On failure, resume from **last checkpoint** (not from scratch)
- Example: **FTP large file transfer** recovery after disconnect

🌐 SESSION LAYER PROTOCOLS

- **NetBIOS** — Windows name resolution & session establishment over TCP/IP
- **RPC (Remote Procedure Call)** — client calls functions on remote server as if local
- **SIP (Session Initiation Protocol)** — VoIP call setup & teardown
- **H.245** — Video conferencing session control
- **PPTP** — VPN session management (legacy)
- **SMB/CIFS** — Windows file sharing sessions (partly L5)

💡 IT ADMIN REAL WORLD

- RDP session disconnected → **session stays on server** until timeout
- Kerberos ticket = **session authentication token**
- "Session expired" in web app = **L5 session timed out**

LAYER 6 — PRESENTATION

Presentation Layer Deep Dive

Job: Translate, encode, compress, and **encrypt data** so it can be understood by the application layer across different systems.

🔄 THREE CORE FUNCTIONS

Translation
ASCII ↔ EBCDIC
Unicode, UTF-8
Character encoding

Compression
GZIP, zlib
JPEG, MPEG
Reduce bandwidth

Encryption
TLS/SSL
AES, RSA
End-to-end secure

📁 FILE FORMAT STANDARDS

- **Images:** JPEG, PNG, GIF, BMP, TIFF, WebP
- **Video:** MPEG, AVI, MP4, H.264, H.265
- **Documents:** PDF, DOCX, HTML (after parsing)
- **Audio:** MP3, AAC, WAV, FLAC

🔐 WHERE SSL/TLS LIVES

- TLS operates **between L5 and L7** — often placed at L6
- Provides: **Confidentiality, Integrity, Authentication**
- Certificate **validates server identity** (and optionally client)
- Current standard: **TLS 1.3** (TLS 1.0/1.1 deprecated)

🖥 PRESENTATION LAYER CLI

```
# Check TLS certificate openssl s_client -connect google.com:443 # Check certificate expiry echo | openssl s_client -connect google.com:443 2>/dev/null | openssl x509 -noout -dates # Verify encoding of a file file document.pdf xxd file.jpg | head -3
```

LAYER 6 — PRESENTATION

TLS/SSL Handshake Deep Dive

1. **ClientHello** — Client sends TLS version, cipher suites, random nonce
2. **ServerHello** — Server picks cipher suite, sends TLS version, random nonce
3. **Certificate** — Server sends X.509 certificate (signed by trusted CA)
4. **Key Exchange** — Client verifies cert, negotiates session key (ECDHE/RSA)
5. **ChangeCipherSpec** — Both sides switch

⚡ TLS 1.3 IMPROVEMENTS

- Only **1 round-trip** for handshake (TLS 1.2 = 2 RTT)
- Removed weak ciphers: **RSA key exchange, RC4, 3DES**
- **0-RTT resumption** for returning clients (faster reconnect)
- Forward secrecy by default: **ECDHE only**

🖥 TLS TROUBLESHOOTING

```
# Test TLS 1.3 support openssl s_client -connect host:443 -tls1_3 # Check certificate chain openssl s_client -showcerts -connect host:443 # Verify cert against CA openssl verify -CAfile ca-bundle.crt cert.pem # Test with curl (verbose) curl -Iv https://example.com 2>&1 | grep -E "SSL|TLS|cert" # Check cipher used nmap --script ssl-cert,ssl-enum-ciphers -p 443 host
```

LAYER 7 — APPLICATION

Application Layer Fundamentals

Job: Provide **network services directly to user applications**. The layer users interact with — browsers, email clients, file transfers.

📋 KEY APPLICATION PROTOCOLS

Protocol	Port	Use	Transport
HTTP	80	Web browsing (unencrypted)	TCP
HTTPS	443	Secure web browsing	TCP
DNS	53	Domain name resolution	UDP/TCP
SMTP	25/587	Email sending	TCP
IMAP	143/993	Email retrieval	TCP
FTP	20/21	File transfer (insecure)	TCP
SFTP/SCP	22	Secure file transfer	TCP
SNMP	161	Network monitoring	UDP

🌐 WHAT HAPPENS AT L7

- Browser sends **HTTP GET** request → L7
- Email client connects to mail server via **SMTP**
- Windows machine queries **DNS** for domain IP
- Admin SSH'd into router → **SSH session** = L7
- SNMP trap from switch to monitoring → **L7**

🔍 L7 DEVICES

- **Application Firewall (WAF)** — inspects L7 traffic
- **Load Balancer** — routes HTTP/HTTPS requests
- **Proxy Server** — intercepts application requests
- **API Gateway** — manages API traffic at L7

LAYER 7 — APPLICATION

DNS Resolution Step by Step

1. User types **www.google.com** in browser. OS checks **local DNS cache** first.
2. Cache miss → OS queries **Recursive Resolver** (usually ISP or 8.8.8.8)
3. Resolver asks **Root Name Server** → returns address of **.com TLD server**
4. Resolver asks **.com TLD server** → returns address of **google.com authoritative NS**

🖥 DNS CLI COMMANDS

```
# Basic DNS lookup nslookup google.com dig google.com host google.com # Look up specific record type dig MX gmail.com
dig AAAA google.com dig TXT google.com # Reverse DNS lookup dig -x 8.8.8.8 # Query specific DNS server dig @8.8.8.8
google.com # Check local DNS cache (Windows) ipconfig /displaydns ipconfig /flushdns
```

LAYER 7 — APPLICATION

HTTP/HTTPS Walkthrough

📤 HTTP REQUEST STRUCTURE

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept:
text/html,application/json
Accept-Encoding: gzip,
deflate, br Connection:
keep-alive Authorization:
Bearer eyJhbGc...
```

📥 HTTP RESPONSE STATUS CODES

2xx
Success: 200 OK, 201 Created

3xx
Redirect: 301 Moved, 302 Found

4xx
Client: 401 Auth, 403 Forbid, 404

5xx

🖥 HTTP/HTTPS CLI TESTING

```
# Full verbose HTTP request curl -Iv https://example.com # Check response headers only curl -I
https://example.com # POST with JSON body curl -X POST https://api.example.com/data \ -H "Content-Type:
application/json" \ -d '{"key":"value"}' # Follow redirects curl -L http://example.com # Measure response time
curl -o /dev/null -s -w "%{time_total} " https://example.com # Capture with Wireshark http.response.code == 200
http.request.method == "POST"
```

ENCAPSULATION & DE-ENCAPSULATION

Data Encapsulation Journey

Each layer **wraps the data** with its own header (and sometimes trailer). This is called **encapsulation**. Receiving side does the reverse.

7	**Application** HTTP/DNS/SMTP data generated	`Data`
6	**Presentation** Encrypt / compress / encode	`Data`
5	**Session** Add session identifiers	`Data`
4	**Transport** Add TCP/UDP header (ports, seq)	`Segment`
3	**Network** Add IP header (src/dst IP, TTL)	`Packet`
2	**Data Link** Add Ethernet header + FCS trailer	`Frame`
1	**Physical** Convert to bits, transmit on wire	`Bits`

📦 SENDER (ENCAPSULATION)

Application generates data → each layer **adds its header** → bits sent on wire

📬 RECEIVER (DE-ENCAPSULATION)

Bits received → each layer **strips its header** → data delivered to application

💡 KEY INSIGHT

- Each layer **only reads its own header** — L3 router ignores L4 TCP header
- A switch at L2 sees **no IP address** — only MAC
- Firewall at L4 sees **port numbers** — can allow/deny by port

COMPARISON

OSI vs TCP/IP Model

OSI Layer	TCP/IP Layer	Protocols
7 Application	Application	HTTP, DNS, SMTP, FTP, SSH
6 Presentation		TLS/SSL, MIME, ASCII
5 Session		NetBIOS, RPC, SIP
4 Transport	Transport	TCP, UDP, SCTP
3 Network	Internet	IP, ICMP, ARP, NAT
2 Data Link	Network Access	Ethernet, 802.11 Wi-Fi
1 Physical		UTP, Fiber, RF signals

🔍 KEY DIFFERENCES

- OSI = **7 layers**; TCP/IP = **4 layers**
- OSI is **conceptual model**; TCP/IP is what the **internet actually uses**
- TCP/IP merges L5+L6+L7 into one **Application layer**
- TCP/IP merges L1+L2 into one **Network Access layer**
- OSI = "should work this way"; TCP/IP = "this IS how it works"

🎯 WHEN TO USE WHICH

- Use **OSI** for: troubleshooting, exams, vendor discussions
- Use **TCP/IP** for: protocol analysis, packet captures, actual implementation

REAL WORLD SCENARIOS

OSI Troubleshooting Scenarios

🔌 Scenario 1

"No network connection"

- **L1:** Check cable seated, link light on?
  ethtool eth0 → Link detected: no?
- **L1:** Try different cable/port
- **L2:** Check NIC driver loaded
  ip link show — is it UP?
- **L3:** DHCP lease received?
  ip addr — any address assigned?
- **L3:** Ping default gateway

Root Cause: Usually L1 (bad cable) or L3 (DHCP failed)

🌐 Scenario 2

"Can ping IP, can't browse"

- **L3:** Ping works → IP connectivity OK
- **L7:** Test DNS first
  nslookup google.com → times out?
- **L7:** DNS failing → check DNS server
  dig @8.8.8.8 google.com
- **L4:** Firewall blocking 53/TCP/UDP?
- **L6:** Try HTTP not HTTPS → cert issue?

Root Cause: Usually L7 DNS or L4 firewall rule

🔒 Scenario 3

"SSL cert error in browser"

- **L6:** Presentation layer issue — TLS/cert
- Check cert expiry date
  openssl s_client -connect host:443
- Check clock on client
  Clock skew breaks cert validation
- Check CA chain complete
  Intermediate cert missing?
- Check CN/SAN matches hostname

Root Cause: Expired cert, clock skew, or missing intermediate CA

QUICK REFERENCE

Golden Rule CLI Commands

L1 — Physical

```
ethtool eth0 # Check link speed & duplex show interfaces Gi0/0 # Physical status + errors
```

L2 — Data Link

```
show mac address-table # CAM table lookup show vlan brief # VLAN list on switch ip neigh show # ARP/neighbor table (Linux)
```

L3 — Network

```
show ip route # Routing table ping 8.8.8.8 # Test L3 reachability traceroute 8.8.8.8 # Hop-by-hop path ip addr show # IP config (Linux)
```

L4 — Transport

```
ss -tunap # All active sockets nc -zv host 443 # Test TCP port netstat -ano # Windows: active connections
```

L5/6 — Session/Pres

```
openssl s_client -connect h:443 # TLS inspect openssl x509 -noout -dates -in c.pem # Cert expiry
```

L7 — Application

```
dig google.com # DNS lookup curl -Iv https://site # HTTP headers + TLS nslookup domain # Windows DNS test nmap -sV host # Service version scan
```

SUMMARY & RECAP

You Now Know OSI!

🧠 MEMORY TRICK

"All People Seem To Need Data Processing"

7 — All	6 — People	5 — Seem	4 — To	3 — Need	2 — Data	1 — Processing
Application	Presentation	Session	Transport	Network	Data Link	Physical

🔑 KEY TAKEAWAYS

- Always troubleshoot **bottom-up** — L1 first, then L2, L3...
- Each layer speaks only to the **layer above and below** it
- Knowing the layer **narrows the fix** immediately
- TCP/IP is what runs the internet — OSI is your **mental model**

Subscribe & Grow!

New episodes every week
CCNA · Networking · Sysadmin

@jahircyberdesk

▶ Next: TCP/IP Deep Dive

📚 RESOURCES

- Free **OSI Study Guide PDF** — comment "OSI PDF"
- **Cisco IOS Lab** — Packet Tracer (free)
- RFC 793 (TCP), RFC 791 (IP), RFC 1122